温州肯恩大学
WENZHOU-KEAN UNIVERSITY

MGS 3001 WHS01
Python Programming for Business

API-Based Data Collection

Dawei Chen

Week 9-1, 21 April 2026

Where We Are

Assignment 3: Data Collection Report

- Due: May 10, 23:59
- Detailed instructions will be updated shortly

Week	Date	Topic	Note
w8	4/14	Research Workshop II	Hypothesis, measurement
	4/16	Intro to Web Data Collection & GitHub Literacy	✓ Done
w9	4/21	API-Based Data Collection	◀ Today
	4/23	Working with JSON & Semi-Structured Data	JSON parsing, flattening, export to CSV
w10	4/28	HTML Scraping & Unstructured Data	HTML structure, requests + BeautifulSoup, text scraping
	4/30	*Research Workshop III (tentative)*	*No lecture; data collection troubleshooting*

Learning Objectives

By the end of this session, you should be able to:

- Explain what an **API** is and how **REST APIs** work
- Understand what a token is, why it is needed, and how it works
- Set up a *GitHub Personal Access Token* for authenticated access
- Apply the 4D AI Fluency framework to scope a data collection task
- Run, inspect, and modify an AI-generated Jupyter notebook that calls the GitHub API

Recap: Two Doors to Web Data

Door 1: API

The Front Door

- Structured, official access
- Returns clean data (JSON)
- Requires authentication

◀ Today's focus

Door 2: Scraping

Reading the Page Yourself

- Parse HTML from web pages
- More flexible but fragile
- Can break when site changes

Session on 4/28

Last session: big picture. ***This session: hands-on with APIs.***

What Is an API?

API = Application Programming Interface [应用程序编程接口]

A defined way for software to request services or data from another software.

A Restaurant Analogy

- **API** = the ordering system
- **Endpoint** = the menu page (URL)
- **Request** = your order submission
- **Parameters** = your order details
- **API Key** = your membership / identity

APIs are everywhere:

- Weather apps pull data from a weather API
- "Sign in with Google" uses Google's authentication API
- Payment systems (Alipay, Stripe) use payment APIs

For data collection:

APIs let you request specific data from a platform in a structured, predictable way.

Types of APIs

By Access (Who can use it?)

- **Public APIs**
  → Available to anyone
  e.g., weather, social media data
- **Private APIs**
  → Used within an organization
  → support internal systems and workflows
- **Partner APIs**
  → Shared with selected external partners
  → enable business collaboration *(e.g., payment integration)*

By Design (How they work)

- **REST APIs** ◀ **Our focus**
  → Use HTTP (GET, POST, etc.)
  → Simple, scalable, widely used
- **SOAP APIs**
  → More rigid, XML-based
  → Used in legacy enterprise systems
- **GraphQL APIs**
  → Flexible queries (request exactly what you need)
  → Efficient for complex data retrieval

REST API is most common for web data collection.

HTTP: How REST APIs Work

Request (what you send)

URL
Where to send the request

Method
GET = retrieve data (most common)
POST = send data to server

Headers
Authentication token, content type

Parameters
Filters: ?q=AI&sort=stars&page=2

Response (what you get back)

Status Code
200 = OK (success!)
404 = Not Found (wrong URL)
403 = Forbidden (need auth)
429 = Too Many Requests (rate limited)

Headers
Rate limit info, content type

Body
The actual data — usually JSON

In Python: `response = requests.get(url, headers=headers, params=params)`

Anatomy of a REST API Request

Example: GitHub Search API

```
https://api.github.com/search/repositories?q=AI-skills&sort=stars&per_page=10
```

Base URL	https://api.github.com	The API's home address
Endpoint	/search/repositories	Which resource you're requesting
Query parameter	Q=AI-skills	What to search for
Sort parameter	sort=stars	How to order results
Pagination	per_page=10	How many results per page

Try it now: paste this URL into your browser. You'll see **raw JSON**.

Reading API Documentation

Before using any API, check its documentation. Look for:

1. **Available endpoints**
   What data can you request? (e.g., /repos, /users, /search/repositories)
2. **Parameters (required vs. optional)**
   Filters, sort options, pagination controls
3. **Authentication requirements**
   API key / token? How to pass it?
4. **Rate limits**
   How many requests per hour/minute?
5. **Example requests & responses**
   Sample JSON output tells you what fields are available.

GitHub REST API documentation

AI can help read and summarize API docs too.

Tokens, Rate Limits & Authentication

What is a token?

A token is like a digital ID card. When you send an API request, the token tells the server who you are. This allows the server to: (1) track how many requests you've made, (2) grant you higher access limits, and (3) control what data you can access.

Think of it as showing your student ID to enter the library — the library needs to know who you are.

Without a token

60 requests / hour

You'll hit the limit in minutes.

With a token

5,000 requests / hour

Free to set up. Takes 2 minutes.

GitHub REST API for searching has its own limit: 30 requests/minute, max 1,000 results per query. Need more? Split by date range.

Setting Up a GitHub Personal Access Token

Why “Personal Access Token” (PAT)?

- A PAT is a long string that acts as a password for API access, but safer.
- Unlike your GitHub password, a PAT can be: **scoped** (only allows specific actions), **expirable** (set to auto-expire), and **revocable** (delete anytime without changing your password).

⚠ **You can only see the token once!**
Save it in a text file on your computer.

Never paste your token directly in code you share or upload to GitHub.

Steps:

1. github.com → click your avatar → Settings
2. Scroll to Developer settings (bottom of left sidebar)
3. Personal access tokens → Tokens (classic) → Generate new token
4. Name it (e.g., “mgs3001_data_collection”)
5. Set expiration (90 days is fine)
6. Select scope: check “public_repo” only
7. Generate → copy the token immediately

Demo: Collecting Data on AI Skills Repositories

From research question to working code — using the 4D framework

Start with the Question, Not the Code

Suppose we want to study the emerging landscape of AI/agent skills on GitHub.

What kinds of skills exist? Which are popular? Who builds them — individuals or companies? How actively are they maintained? Are they open for commercial use?

The first step is NOT coding.

It's defining:

- What data do I need?
- How do I find it? (which keywords? which filters?)
- What does the final dataset look like?

Data collection begins with a question, not a prompt.

How Would a Human Search?

Choosing the right search terms determines what data you get. This is the **scoping problem**.

Before any code, let's do what a human would do:

- Search "AI skills" on GitHub — how many results?
- Search "agent skills" — different results?
- Search "agent-skill" (with dash) — different again?
- Search "AI skill" (singular) vs. "AI skills" (plural) — do you miss results?

What we discover:

- **Case-sensitive?** No — GitHub search is case-insensitive
- **Space / dash sensitive?** Yes — "agent-skills" as a topic tag ≠ "agent skills" as keywords
- **Plural sensitive?** Partially — different result counts for singular vs. plural

→ Try GitHub's Advanced Search to see filter options (language, stars, date…)

From Manual to Automated

What a human does manually

- Search “AI skills” on GitHub
- Open each repository
- Read: name, description, stars, forks…
- Copy to a spreadsheet
- Repeat 200 times

Hours of tedious work

What Python does (same process!)

- Send search request to API
- Read the JSON response
- Extract: name, description, stars…
- Save to a DataFrame
- Loop through all pages automatically

⚡ Done in minutes

The code mirrors what you would do manually.

So you need to know what a human would do first — what to search, what to collect, how to **scope** it.

Don’t Just Prompt — Apply the 4D Framework

❌ **Bad prompt:** *“Write Python code to collect data on AI skills repositories in GitHub”*

Which keywords? What fields? How many results? What filters? The AI will guess.

Instead, use the 4D framework to think before prompting:

- **Delegation:** *What is my goal? What does the final dataset need to look like?*
- **Description:** *Define keywords, fields, filters, output format explicitly before prompting*
- **Discernment:** *Does the code search the right terms? Collect all the fields I need?*
- **Diligence:** *Am I respecting rate limits? Is my search scope defensible?*

Good prompts come from good thinking, not prompt engineering tricks.

Defining the Dataset

API Field	What It Tells Us
full_name	Identity of the skill/project
description	What the skill does (text analysis later)
stargazers_count	Market attention / demand proxy
forks_count	Adoption / reuse signal
open_issues_count	User engagement / quality signal
created_at	Entry timing (when was it created?)
updated_at	Maintenance activity (is it still alive?)
license	Commercial openness
owner.type	Producer type (individual vs. organization)
topics	Self-declared categories (unstructured)

Each field is not just “a column” — it’s a potential variable for analysis.

This mapping connects data collection to research design — every field serves a purpose.

An Example of Prompt

✓ **Built from the 4D thinking process:**

```
Create a Jupyter notebook that uses the GitHub REST API
(/search/repositories endpoint) to collect data on AI
skills
and agent skills repositories.

Search using multiple keywords: 'AI skills', 'AI skill',
'agent skills', 'agent-skills', 'claude skills', 'LLM
skills'.

For each repo, collect: full_name, description,
stargazers_count, forks_count, open_issues_count,
created_at, updated_at, license name, owner login,
owner type, topics.

Handle pagination (up to 200 results per keyword).
Use a personal access token. Deduplicate across keywords.
Export to CSV.
```

vs. the bad prompt:

Write Python code to collect data on AI skills repositories in GitHub

What changed:

specific keywords, explicit fields mapped to research constructs, deduplication, clear output.

Live Demo: Walking Through the Code

week9-1_github-api_collection.ipynb

In VS Code, walk through the AI-generated notebook cell by cell:

Cell 1	Import libraries (requests, pandas, json, time)
Cell 2	Set up token and HTTP headers for authentication
Cell 3	Define keywords list and API URL
Cell 4	Make the first API call, inspect the raw JSON
Cell 5	Parse JSON: extract fields from each repo
Cell 6	Loop through keywords + pagination
Cell 7	Deduplicate results across keywords
Cell 8	Convert to DataFrame, preview, export to CSV

Discernment check:

What did the AI do well? What would you change or add?

Understanding the JSON Response

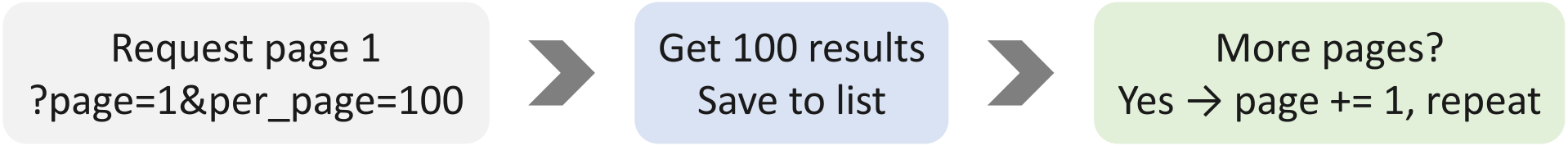
```
{ "total_count": 2456,
  "items": [
    { "full_name": "anthropics/courses",
      "stargazers_count": 8200,
      "forks_count": 1100,
      "language": "Python",
      "created_at": "2024-05-15T...",
      "description": "Anthropic's educational...",
      "owner": { "login": "anthropics",
                 "type": "Organization" },
      "license": { "name": "MIT" },
      "topics": ["ai", "skills", "claude"]
    }, ...
  ]
}
```

Nested objects (owner, license) and arrays (topics). We'll handle these properly on Thursday.

Pagination: Collecting Beyond One Page

Why pagination?

APIs don’t return all results at once. They send them in pages (e.g., 100 per page).




```
for page in range(1, max_pages + 1):
    params['page'] = page
    response = requests.get(url,
headers=headers, params=params)
    items = response.json()['items']
    all_repos.extend(items)
```

GitHub search limit: max 1,000 results per query.

For more: split queries by date range.

Hands-on Task

Your turn! (remaining class time)

Task 1: Run the demo notebook as-is. Inspect the output CSV.

Task 2: Change the keywords (e.g., add “workflow automation”, “MCP skills”, or terms related to your research topic).

Task 3: Add or remove fields (e.g., add “has_wiki”, “size”).

Task 4 (stretch): Add date filters (e.g., created:>2024-01-01) or sort by different criteria.

Stuck? Check your token setup first. Then ask AI. Stuck? Raise your hand.

Common Issues & Troubleshooting

401 Unauthorized → Token is wrong, expired, or not sent. Check headers.

403 Forbidden → Rate limit exceeded. Wait, or check token is applied.

422 Unprocessable → Bad query syntax. Check search parameters.

Notebook won't run → Check Python kernel. Try: Kernel → Restart.

Empty results → Check query spelling. Try broader terms.

KeyError in JSON → Field doesn't exist for some repos (e.g., license is null). Add error handling.

Most issues: (1) Is my token correct? (2) Am I sending it in headers?

What We Built Today

A reusable data collection pipeline:

Today's key lesson:

- Data collection begins with thinking, not coding.
- The first two steps (***Define Scope*** + ***Craft Prompt***) matter as much as the coding.
- The code mirrors your thinking. Better thinking → better code → better data.

Does Your Platform Have an API?

Check before you scrape! Many platforms offer APIs:

How to check:

- Google: “[platform name] API documentation”
- Look for a “Developers” or “API” link in the platform’s footer
- Search GitHub for Python wrapper libraries
- Ask AI: “Does [platform] have a public API for data collection?”

Platforms with well-documented APIs:
GitHub, Reddit, Spotify, Twitter/X, Stack Overflow, OpenWeather, World Bank, various government portals

If an API exists, it’s almost always the better option. If not → HTML scraping (4/28).

Next Session: JSON & Semi-Structured Data (4/23)

What we'll cover on Thursday:

- What is JSON and how to read its structure
- Handling nested objects (e.g., owner.login, license.name)
- Flattening semi-structured data into clean tabular format
- Dealing with missing values and inconsistent data in JSON
- Building a research-ready CSV from raw API output

Bring: today's notebook and the CSV you generated. We'll build on them.

Q&A

dchen@kean.edu CBPM B214

Office hours: Tue & Thu 11:15–12:30 | Wed 14:00–16:30